Kamil Suwiczak 160398, 14.04.2026

Sprawozdanie z szyfrów blokowych

1) Porównanie tryby pracy szyfrów blokowych

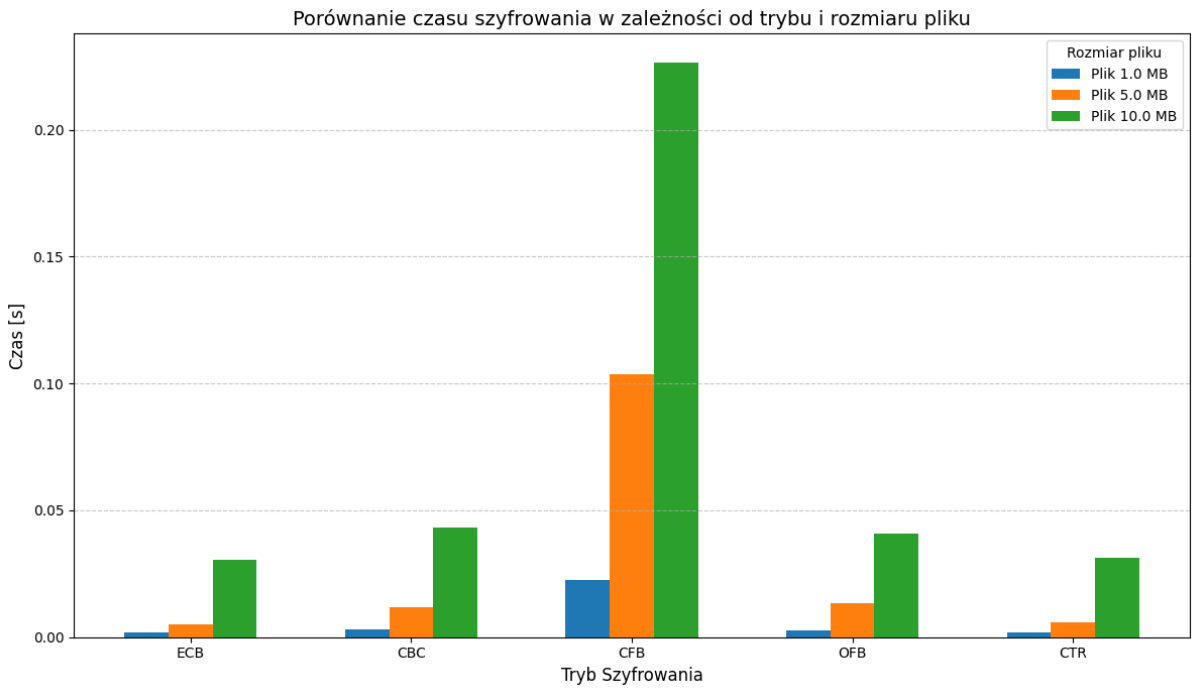
Tryb	Czas szyfrowania [s]	Czas deszyfrowania [s]	Rozmiar pliku [B]
ECB	0.001727	0.001503	1023919
ECB	0.005031	0.005413	5119599
ECB	0.030624	0.009539	10239199

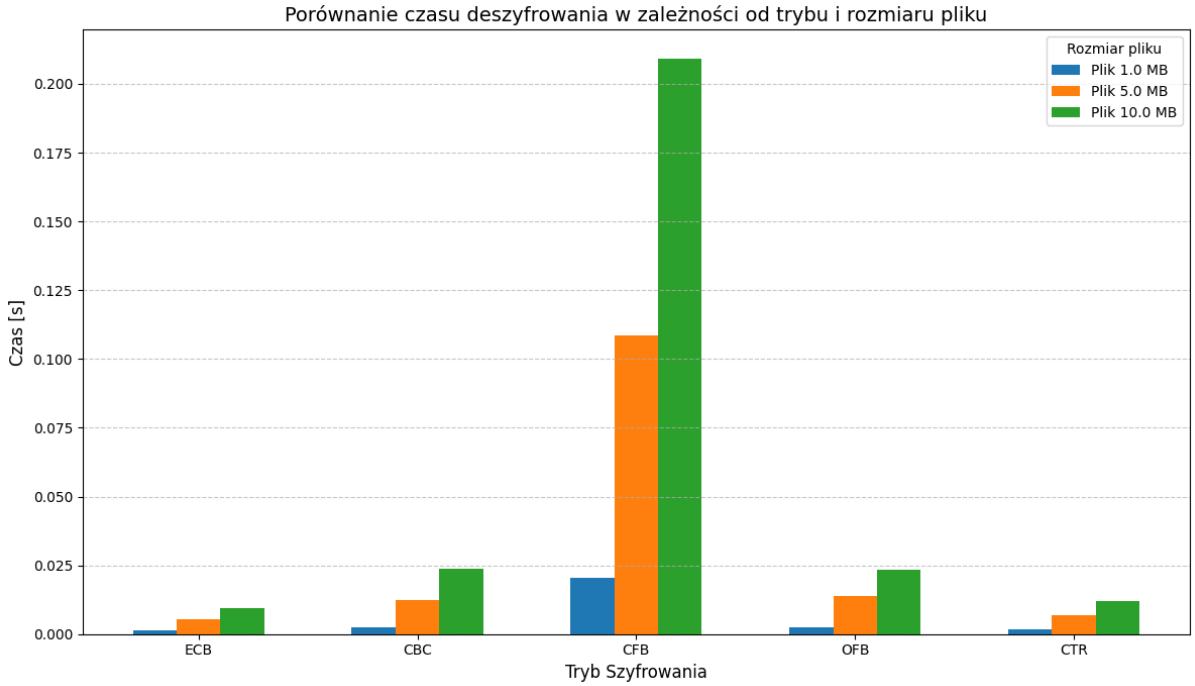
Tryb	Czas szyfrowania [s]	Czas szyfrowania [s]	Rozmiar pliku [B]
CBC	0.002923	0.002517	1023919
CBC	0.011847	0.012301	5119599
CBC	0.043410	0.023924	10239199

Tryb	Czas szyfrowania [s]	Czas szyfrowania [s]	Rozmiar pliku [B]
OFB	0.002662	0.002646	1023919
OFB	0.013497	0.013872	5119599
OFB	0.040819	0.023346	10239199

Tryb	Czas szyfrowania [s]	Czas szyfrowania [s]	Rozmiar pliku [B]
CFB	0.022554	0.020479	1023919
CFB	0.103837	0.108443	5119599
CFB	0.226541	0.209227	10239199

Tryb	Czas szyfrowania [s]	Czas szyfrowania [s]	Rozmiar pliku [B]
CTR	0.001939	0.001791	1023919
CTR	0.005742	0.006920	5119599
CTR	0.031257	0.011831	10239199





Tryb CFB zarówno w procesie szyfrowania, jak i deszyfrowania jest kilkukrotnie wolniejszym trybem niż pozostałe. Wynika to z faktu, że tryb CFB operuje na mniejszych partiach danych i każdy blok musi czekać na zakończenie szyfrowania poprzedniego, co skutkuje brakiem możliwości zrównoleglenia.

Pozostałe tryby są znacznie szybsze, a najszybszym z nich jest tryb ECB. Ponadto można zauważyć, że czas szyfrowania i deszyfrowania rośnie liniowo w zależności od wielkości pliku.

2) Analiza propagacji błędów
   a) Tryb ECB – błąd w jednym bloku szyfrogramu wpływa jedynie na odpowiadający mu blok tekstu jawnego, w skutek tego nieczytelny jest tylko mały fragment tekstu jawnego równy jednemu blokowi
   b) Tryb CBC – błąd w jednym bloku szyfrogramu wpływa na 2 bloki tekstu jawnego (odpowiadający mu oraz następny), w skutek tego nieczytelny jest tylko mały fragment tekstu jawnego równy dwóm blokom
   c) Tryb CFB – wystąpienie błędu w jednym bloku powoduje, że uszkodzeniu ulega fragment wiadomości jawnej tak długo, aż uszkodzony bit opuści rejestr wejściowy, tryb ten jest samo synchronizujący się po pewnym czasie
   d) Tryb OFB – błąd w szyfrogramie na pozycji k powoduje błąd w tekście jawnym jedynie na pozycji k
   e) Tryb OFB – błąd w szyfrogramie na pozycji k powoduje błąd w tekście jawnym jedynie na pozycji k

3) Implementacja trybu CBC przy pomocy trybu ECB

```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import os

def xor_bytes(b1, b2):
    return bytes(a ^ b for a, b in zip(b1, b2))

def manual_cbc_encrypt(plaintext, key, iv):
    padded_data = pad(plaintext, AES.block_size)

    ecb_cipher = AES.new(key, AES.MODE_ECB)

    ciphertext = b''
    previous_block = iv

    for i in range(0, len(padded_data), AES.block_size):
        current_plaintext_block = padded_data[i:i+AES.block_size]

        xored_block = xor_bytes(current_plaintext_block, previous_block)

        encrypted_block = ecb_cipher.encrypt(xored_block)

        ciphertext += encrypted_block
        previous_block = encrypted_block

    return ciphertext
```

```python
def manual_cbc_decrypt(ciphertext, key, iv):
    ecb_cipher = AES.new(key, AES.MODE_ECB)

    decrypted_padded_data = b''
    previous_block = iv

    for i in range(0, len(ciphertext), AES.block_size):
        current_ciphertext_block = ciphertext[i:i+AES.block_size]

        decrypted_block = ecb_cipher.decrypt(current_ciphertext_block)

        plaintext_block = xor_bytes(decrypted_block, previous_block)

        decrypted_padded_data += plaintext_block
        previous_block = current_ciphertext_block

    return unpad(decrypted_padded_data, AES.block_size)
```

```python
if __name__ == "__main__":
    key = b'1234567890123456'
    iv = os.urandom(16)
    message = b"Wiadomosc do zaszyfrowania"

    print("Oryginalna wiadomość:", message)

    my_ciphertext = manual_cbc_encrypt(message, key, iv)
    print("Szyfrogram:", my_ciphertext.hex())

    my_decrypted = manual_cbc_decrypt(my_ciphertext, key, iv)
    print("Zdeszyfrowana wiadomość:", my_decrypted.decode('utf-8'))
```